Lekcja 10

Definiowanie klas

Klasa jako złożony typ danych: pola, metody, dostęp i obiekty

Cel lekcji

- zdefiniować klasę w języku C++ z użyciem składni class
- odróżnić zmienne członkowskie od funkcji członkowskich
- wyjaśnić działanie specyfikatorów dostępu public, private i protected
- zadeklarować zmienną obiektową oraz przygotować prostą klasę z metodami

Materiał: technik programista, klasa 4, przedmiot Programowanie aplikacji, kwalifikacja INF.04.

Słownik na start

Nowe skróty i pojęcia

OOP / POO

programowanie zorientowane obiektowo: klasy, obiekty i komunikacja między nimi.

class

słowo kluczowe C++ rozpoczynające definicję klasy.

public

elementy dostępne z zewnątrz klasy, np. przez obiekt.

private

elementy dostępne tylko wewnątrz klasy; domyślny dostęp w class.

protected

elementy dostępne w klasie i klasach potomnych; ważne przy dziedziczeniu.

::

operator zakresu, wskazuje, do której klasy należy metoda definiowana poza klasą.

Co opisuje klasa?

Klasa jako typ złożony



Najważniejsza myśl

Klasa nie jest jeszcze obiektem. Klasa jest projektem typu, a obiekt jest konkretnym egzemplarzem utworzonym na podstawie klasy.

Schemat z podręcznika

Ogólna postać definicji klasy

Składnia class w C++

```
class nazwa_klasy {
    specyfikator_dostepu_1:
        definicje_zmiennych_czlonkowskich_1;
        deklaracje_funkcji_czlonkowskich_1;
        definicje_funkcji_czlonkowskich_1;

    specyfikator_dostepu_2:
        definicje_zmiennych_czlonkowskich_2;
        deklaracje_funkcji_czlonkowskich_2;
        definicje_funkcji_czlonkowskich_2;
};
```

- nazwa_klasy jest identyfikatorem nowego typu
- specyfikator dostępu ustala widoczność elementów
- pola przechowują dane, a metody wykonują operacje
- definicja klasy kończy się średnikiem po nawiasie klamrowym

public, private, protected

Specyfikatory dostępu

public

elementy dostępne z każdego miejsca, w którym widoczny jest obiekt klasy

private

elementy dostępne tylko wewnątrz tej samej klasy; domyślne dla class

protected

elementy dostępne w klasie i w klasach pochodnych; pełne znaczenie przy dziedziczeniu

Dobra praktyka

Dane najczęściej ukrywa się jako private, a dostęp do nich zapewnia przez publiczne metody.

Przykład idei hermetyzacji

```
class Konto {
private:
    double saldo;
public:
    void wplac(double kwota);
```

Pola przechowują stan obiektu

Zmienne członkowskie klasy

- reprezentują dane zapisywane w obiektach będących instancjami klasy
- mogą mieć typy proste, np. int, float, string
- mogą mieć także typy złożone, np. tablice, struktury lub inne klasy
- są nazywane także danymi członkowskimi albo polami

Przykład pól klasy

```
class Uczen {
public:
    string imie;
    string nazwisko;
    string klasa;
};
```

Każdy obiekt klasy Uczen będzie miał własne wartości pól: imie, nazwisko i klasa.

Metody opisują zachowanie obiektu

Funkcje członkowskie klasy

Funkcja członkowska

Metoda jest funkcją należącą do klasy. Może korzystać z pól obiektu i wykonywać na nich operacje.

Dwa warianty definicji

Metodę można zdefiniować wewnątrz klasy albo tylko zadeklarować w klasie, a definicję umieścić poza klasą.

Definicja metody w klasie

```
class Pracownik {
public:
    string imie, nazwisko;

    void wyswietlDane() {
        cout << "Dane pracownika: "
             << imie << " " << nazwisko;
    }
};
```

Operator zakresu ::

Definicja metody poza klasą

W klasie: deklaracje metod

```
class Pracownik {
public:
    string imie, nazwisko;
    void ustawImie(string);
    void ustawNazwisko(string);
    void wyswietlDane();
};
```

Poza klasą: definicje metod

```
void Pracownik::ustawImie(string pImie) {
    imie = pImie;
}

void Pracownik::ustawNazwisko(string pNazwisko) {
    nazwisko = pNazwisko;
}
```

Zapamiętaj

Zapis Pracownik::ustawImie oznacza: funkcja ustawImie należy do klasy Pracownik. Operator :: nazywa się operatorem zakresu.

Od Pracownik do Uczen

Przykład lekcyjny: klasa Uczen

Deklaracja klasy

```
class Uczen {
public:
    string imie, nazwisko, klasa;

    void ustawImie(string pImie);
    void ustawNazwisko(string pNazwisko);
    void ustawKlase(string pKlasa);
    void wyswietlDane();
    void wyswietlPersonalia();
    void wyswietlKlase();
};
```

Przykładowe definicje metod

```
void Uczen::ustawKlase(string pKlasa) {
    klasa = pKlasa;
}

void Uczen::wyswietlPersonalia() {
    cout << imie << " " << nazwisko;
}

void Uczen::wyswietlKlase() {
    cout << klasa;
}
```

Jak utworzyć obiekt klasy?

Deklarowanie zmiennych obiektowych

Zasada

Aby korzystać z obiektu, trzeba go wcześniej utworzyć jako instancję określonej klasy.

Sposób 1: po definicji klasy

```
class Pracownik {
    ...
} pracownik1, pracownik2;
```

Sposób 2: jak zwykłą zmienną

```
Pracownik pracownik;
Uczen uczen1, uczen2;
```

Ważne

W dalszych lekcjach obiekty będą tworzone także z użyciem konstruktora. Konstruktor to specjalna metoda uruchamiana podczas tworzenia obiektu.

Na co uważać?

Typowe błędy przy definiowaniu klas

- brak średnika po zamknięciu definicji klasy: };
- próba użycia pola private bezpośrednio z zewnątrz klasy
- definicja metody poza klasą bez operatora zakresu ::
- mylenie klasy z obiektem: class Uczen to typ, a Uczen jan; to obiekt
- zbyt szerokie publiczne udostępnienie danych, które powinny być ukryte

Dobra klasa ma czytelną nazwę, przemyślane pola, metody i odpowiednio dobrany dostęp.

Co uczeń powinien umieć?

Podsumowanie lekcji

- zapisać podstawową składnię klasy w C++
- wskazać pola i metody w definicji klasy
- wyjaśnić znaczenie public, private i protected
- zdefiniować metodę wewnątrz klasy i poza klasą z użyciem ::
- zadeklarować obiekt klasy oraz opisać różnicę między klasą a obiektem

Następny krok: deklarowanie obiektów, konstruktory i praktyczne użycie klas w programie.